

Client-centric consistency in Apache Cassandra

Replicated database experiments under normal operation, node failure and network partition

[Course name]

[Member 1 name / student ID]

[Member 2 name / student ID, if applicable]

[Member 3 name / student ID, if applicable]

Submission deadline: 27 September 2026 | Group size: at most three

Measured run: 20260912T063741Z | 5 repetitions per matrix cell

Abstract

We deployed three Cassandra 5.0.9 nodes and tested five write/read consistency combinations against four client-centric properties. The main matrix contains 300 trials: 29 violation witnesses, 179 completed trials without a witness, and 92 inconclusive trials. Inconclusive outcomes include unavailable or timed-out operations; they are not counted as consistency failures.

The experiment isolates the effects of replica divergence, quorum requirements and read repair. Separate controls change the read-repair setting and deliberately reverse mutation timestamps. Results apply to the specified sequential workloads and fault schedules; they do not prove a general causal-consistency guarantee for Cassandra.

All result tables are generated from saved database responses. Predictions are preserved separately in `report/predictions.md`. The reproduction archive contains code, environment metadata, initialization records and operation histories.

1. Database and deployment

Cassandra is a replicated, partitioned database in which any node can coordinate a request. Our keyspace uses NetworkTopologyStrategy with dc1:3. With exactly three nodes, every test key has a replica on every node. ONE, QUORUM and ALL require respectively one, two and three replica responses in this deployment. [2]

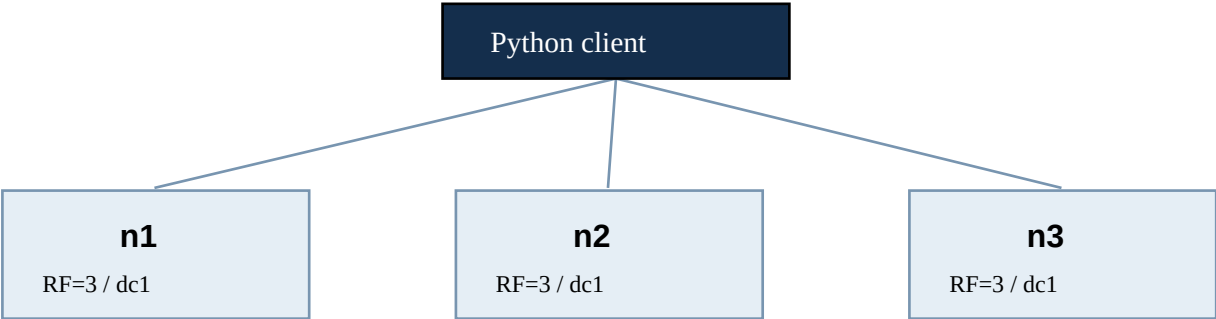


Figure 1. One Docker bridge network. The client reaches all CQL endpoints on port 9042. A partition blocks storage traffic on ports 7000/7001 between selected nodes, while CQL remains reachable. No host ports are published.

Component	Measured / configured value
Database	Apache Cassandra 5.0.9; three containers; 16 tokens per node
Client / JVM	Python 3.11.13; cassandra-driver 3.29.2; protocol v4; openjdk 17.0.20 2026-07-21
Docker	20.10.23; Docker Compose version v2.15.1; aarch64; 4 virtual CPUs
Memory	Docker VM: 7.67 GiB; Cassandra heap: 768 MiB/node
Replication	NetworkTopologyStrategy; dc1=3; rack1; persistent named volumes
Repair controls	Hints disabled; no scheduled anti-entropy repair; BLOCKING unless specified
Query controls	Explicit CL and timestamps; no driver retries or speculative execution; table speculative_retry=NONE

Installation uses the Docker Official Image packaging [7], with pinned base-image digests. A derived image installs iptables and disables hinted handoff. Run python3 scripts/lab.py up to build and start nodes sequentially; readiness checks require three UN nodes and working CQL/schema operations. Exact runtime/image details are saved in environment.json and initial_cluster.json.

2. Models and predictions

The four session guarantees describe the view and ordering experienced by one logical client, even when it changes servers. RYW requires later reads to include that client's earlier writes; MR disallows losing previously observed writes. MW orders one client's writes. WFR preserves dependencies from an observed write to a later write by the reader. [1]

Our checks use a monotone cell a for reads, and distinct cells a (predecessor) and b (successor) for dependencies. We use one logical writer per key, sequential operations, no deletions or TTLs, and increasing timestamps except in the explicit timestamp control. These assumptions make returned values interpretable as evidence about the intended history.

Write/read	RYW	MR (BLOCKING)	MW / WFR
ONE/ONE	May fail	May fail	May expose missing dependency
QUORUM/ONE	May fail	May fail	No general replica-order guarantee
ONE/QUORUM	May fail	Expected for successful quorum reads	No general replica-order guarantee
QUORUM/QUORUM	Expected under assumptions	Expected for successful quorum reads	No general replica-order guarantee
ALL/ALL	Expected under assumptions	Expected for successful reads	No completed witness expected in this workload

These are predictions, not observations. For RF=3, W+R>3 ensures acknowledgement-set intersection; ONE/QUORUM has 1+2=3 and lacks that argument. The overlap argument supports our RYW prediction for QUORUM/QUORUM, but does not order every mutation at every replica. [2; deduction for this workload]

BLOCKING read repair is documented to provide monotonic quorum reads; NONE reconciles a read without repairing divergent replicas. [3] The focused control therefore predicts a possible 1-to-0 regression only with NONE. The main partition schedule may prevent a quorum read from completing at all, which is an availability result rather than evidence of a regression.

For normal operation we expect mostly successful histories. With n3 killed, ONE and QUORUM should remain usable through n1/n2, whereas ALL should fail. With {n1,n2} isolated from {n3}, weak isolated reads can be stale; isolated quorum operations should fail. report/predictions.md contains the complete pre-execution reasoning.

3. Experimental design

Each of five configurations is tested under three scenarios for four models, with 5 fresh-key repetitions: $5 \times 3 \times 4 \times 5 = 300$ main trials. Each row is initialized to $(a=0, b=0)$ at ALL before the fault is injected. Initialization errors abort the run. Writes use explicit increasing microsecond timestamps to separate replication effects from clock effects. CQL supports the `TIMESTAMP` parameter. [5]

Model	Ordered operations	Violation witness
RYW	W(a=1) at n1; R(a,b) at target	Final $a=0$ after acknowledged W
MR	W(a=1) at n1; R at n1; R at target	Second read $a <$ first read a
MW	W(a=1) at n1; W(b=1) at target; R at target	Final $(a=0, b=1)$
WFR	Producer W(a=1) at n1; client R(a=1) at n1; client W(b=1) at target; observer R at target	Final $(a=0, b=1)$, with dependency read confirmed

Target is n3 for normal operation and partition, and n2 for node failure. W and R use the selected configuration. The WFR producer is logically distinct from the reader/writer even though operations use the same harness. The MW/WFR observer selects both cells together; separate reads could themselves introduce a misleading observation order. A visible successor with its predecessor absent is a counterexample to dependency visibility, but absence of a witness cannot establish every replica's application order.

Node failure sends `SIGKILL` to n3, then allows 15 seconds for failure detection. Network partition installs symmetric packet-drop rules for n3 versus n1/n2 and also waits 15 seconds. Rules match both source and destination storage ports to block established streams in both directions. New restrictions are installed before obsolete ones are removed during topology changes.

The driver pins each connection to its named coordinator and propagates errors instead of retrying or downgrading CL. [6] Coordinator pinning does not generally force a local replica read. During isolation, however, a successful ONE read on n3 cannot obtain data from n1/n2. The two-node read-repair control forces both available replicas to participate in a quorum.

Every trial retains the requested CL, query, parameters, nanosecond start/end times, latency, coordinator, value or error. Any operation error makes the trial inconclusive; WFR also requires its dependency read to observe $a=1$. A successor not observed is likewise inconclusive. Fault rules and packet counters are preserved separately.

4. Main results: consistency witnesses

Each cell is V / N / I: violation witnesses / no violation observed / inconclusive trials. N describes a finite observation, not a proven guarantee. The table is computed from trials.json.

Scenario	Write/read	RYW	MR	MW	WFR
normal	ONE/ONE	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
normal	QUORUM/ONE	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
normal	ONE/QUORUM	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
normal	QUORUM/QUORUM	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
normal	ALL/ALL	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
node failure	ONE/ONE	0 / 4 / 1	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
node failure	QUORUM/ONE	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
node failure	ONE/QUORUM	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
node failure	QUORUM/QUORUM	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0	0 / 5 / 0
node failure	ALL/ALL	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5
partition	ONE/ONE	4 / 0 / 1	5 / 0 / 0	5 / 0 / 0	5 / 0 / 0
partition	QUORUM/ONE	5 / 0 / 0	5 / 0 / 0	0 / 0 / 5	0 / 0 / 5
partition	ONE/QUORUM	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5
partition	QUORUM/QUORUM	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5
partition	ALL/ALL	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5	0 / 0 / 5

Across the matrix: 29 V, 179 N and 92 I. Counts are not estimates of production anomaly probability: fresh keys repeat a deliberately selected schedule on one shared cluster, in a fixed configuration order.

Example witness: partition, ONE/ONE, RYW, repetition 1. Key: 20260912T063741Z:partition:ONE/ONE:RYW:1

n1 ONE: UPDATE -> write acknowledged (ok)

n3 ONE: SELECT -> {'a': 0, 'b': 0} (ok)

5. Availability and latency

An operation can fail to obtain enough replicas without returning stale data. The table separates successful requests from errors. Latency is measured at the Python application and includes initial connection setup where applicable; these figures are diagnostic, not a throughput benchmark. p95 uses the nearest-rank definition.

Scenario	Write/read	OK / total	OK %	p50 ms	p95 ms
normal	ONE/ONE	60/60	100.0%	2.3	5.2
normal	QUORUM/ONE	60/60	100.0%	2.1	4.3
normal	ONE/QUORUM	60/60	100.0%	2.4	5.9
normal	QUORUM/QUORUM	60/60	100.0%	2.2	4.2
normal	ALL/ALL	60/60	100.0%	2.5	5.2
node failure	ONE/ONE	59/60	98.3%	1.6	8.1
node failure	QUORUM/ONE	60/60	100.0%	1.7	2.1
node failure	ONE/QUORUM	60/60	100.0%	1.6	2.5
node failure	QUORUM/QUORUM	60/60	100.0%	1.7	2.3
node failure	ALL/ALL	0/60	0.0%	-	-
partition	ONE/ONE	59/60	98.3%	1.8	6.5
partition	QUORUM/ONE	50/60	83.3%	2.0	4.2
partition	ONE/QUORUM	40/60	66.7%	1.9	3.6
partition	QUORUM/QUORUM	30/60	50.0%	2.1	3.8
partition	ALL/ALL	0/60	0.0%	-	-

Recorded error categories: ReadTimeout: 2, Unavailable: 180.

A write timeout is an indeterminate outcome: some replicas may already contain the mutation. The harness does not assume failed writes were rolled back, and does not retry them. Even if later operations succeed, the affected model trial remains inconclusive. Fresh keys prevent such partial writes from contaminating later trials.

6. Read-repair and timestamp controls

Read repair: seed a minority-only version by writing `a=1` on isolated `n3` at `ONE`. Permit only `{n2,n3}` to communicate and read at `QUORUM` through `n3`; then permit only `{n1,n2}` and read at `QUORUM` through `n1`. Repeat with each table setting. Hints remain disabled, and transitions never temporarily reconnect all three nodes.

Table setting	Observed read sequences and counts	Trials
blocking	1 -> 1: 5	5
no_repair	1 -> 0: 5	5

Observed regressions: `BLOCKING=0`, `NONE=5`. This matches the predicted contrast.

The intended difference is whether the first quorum read carries the minority value onto `n2`, the intersection member of the next quorum. This control deliberately uses an initial `ONE` write and changes network topology between quorum reads. It is distinct from the main fixed-topology matrix and tests the documented read-repair mechanism. [3]

Timestamp control: initialize at `T`, write `a=1` at `T+100` through `n1` using `ALL`, then write `a=2` at `T+50` through `n2` using `ALL`, and read through `n3` using `ALL`. This models a later request carrying an older timestamp; it does not change the machines' clocks.

Step	Status	Returned value / effect
Write <code>a=1</code> at <code>T+100</code>	ok	Acknowledgement only
Write <code>a=2</code> at <code>T+50</code>	ok	Acknowledgement only
Read at <code>ALL</code>	ok	<code>{'a': 1, 'b': 0}</code>

Cassandra resolves conflicting cell values by mutation timestamp. [2] Thus a later acknowledged request need not win if its supplied timestamp is lower. This control tests the boundary of our increasing-timestamp assumption; it is excluded from the main violation counts.

7. Discussion, limitations and AI disclosure

The measurements distinguish consistency violations from loss of availability. Weak reads exposed replica divergence; quorum requirements instead made isolated operations unavailable. Successful quorum reads in the focused control address a different question: whether an earlier observed minority version remains visible after the read quorum changes.

Normal operation produced 100 trials without a witness and 0 witnesses. Node failure produced 79 trials without a witness and 21 inconclusive trials. A ReadTimeout at ONE shows that the surviving replica count alone did not ensure every request completed; replica selection or detection timing may contribute, but the trace does not establish the cause.

The partition produced 29 witnesses (ONE/ONE: RYW, MR, MW, WFR; QUORUM/ONE: RYW, MR). Stale reads and missing dependencies demonstrate the predicted risks where observed. Isolated operations that require two or three replicas cannot meet that requirement. Their inconclusive trials do not empirically establish the corresponding session guarantees.

Quorum intersection is insufficient to claim all four guarantees in every execution. Our dependency checks search for a specific out-of-order visibility witness. They do not observe replica execution logs, cross-partition dependencies, concurrent writers, or all possible schedules. ALL/ALL results likewise concern completed operations under the test assumptions, not arbitrary timestamp choices or failed writes.

The deployment shares one host, one network bridge and one datacenter. Resource contention, pauses and connection establishment affect timing. The 5 fixed-schedule repetitions per case cannot establish universal correctness or production failure rates. The failure scenario tests one killed node and routes subsequent work to survivors; it does not measure automatic client failover or permanent loss of storage.

Hints are normally a catch-up mechanism for unavailable replicas. [4] We disable them to retain controlled divergence and perform no background anti-entropy repair. Consequently the measurements do not estimate convergence time with default settings. Future work should repeat with hints enabled, inject latency and packet loss, vary timestamps and concurrent writers, inspect replica histories, and expand the number of machines and trials.

AI usage: OpenAI Codex assisted with experiment design, source-code creation, documentation lookup, local execution, result analysis, report generation and verification on 12 September 2026. [8] Report tables are derived from saved responses of the actual Cassandra containers; no synthetic outcomes are substituted. Group members should review and understand the code, interpretations and citations before submission. Names and course details remain editable in report/authors.json.

8. Reproduction and references

Run `python3 scripts/lab.py up`, then `python3 scripts/lab.py run --repeats 5`. Install `requirements-report.txt` (ReportLab 4.4.3 and PyMuPDF 1.26.4) in a virtual environment and run `scripts/build_report.py` with that interpreter. The report generator selects the latest completed run; use `--results results/<run>` to select this one explicitly. Run `python3 scripts/lab.py down` afterward to release container resources while preserving volumes.

Use `scripts/lab.py heal` after an interruption. Review `README.md` for prerequisites, the macOS Compose executable override, file descriptions and cleanup. Unit tests exercise stale reads, read regression, dependency witnesses and inconclusive errors. A completion marker is required before a run can be used for the report.

Included run directory: `results/20260912T063741Z`. Detailed deployment status, raw initialization writes, fault evidence, operation histories and both controls are retained. The generated `submission.zip` contains these records and the source files. `SHA256SUMS.txt` identifies the archived files.

References below were accessed on 12 September 2026. Cassandra documentation uses the stable branch; the installed server release and Docker digests are recorded independently.

- [1] Terry, D. B., Demers, A. J., Petersen, K., Spreitzer, M. J., Theimer, M. M., and Welch, B. B. (1994). Session Guarantees for Weakly Consistent Replicated Data. *PDIS*, pp. 140-149. [Source](#)
- [2] Apache Cassandra. Dynamo: replication, versioning and tunable consistency. [Source](#)
- [3] Apache Cassandra. Read repair. [Source](#)
- [4] Apache Cassandra. Hints. [Source](#)
- [5] Apache Cassandra. CQL Data Manipulation: UPDATE and TIMESTAMP. [Source](#)
- [6] Apache Cassandra Python driver. Policies implementation. [Source](#)
- [7] Docker Official Image packaging. Cassandra. [Source](#)
- [8] OpenAI. Codex. AI assistance used for this project; disclosure on page 8. [Source](#)